VIT Bhopal University

VITyarthi - Build Your Own Project

LOAN APPROVAL PREDICTOR

A Machine Learning Based Loan Approval Prediction System

Course: **Fundamentals in AIML**

Submitted by: **Mohit Thakur**

Registration No.: **25MIM10126**

GitHub: github.com/mohit939926-a11y/AI-ML-vityarthi-project

October 2026

1. Introduction

Banks and financial institutions receive thousands of loan applications every day. Manually checking each application for income, credit history, loan amount and other details is slow, expensive and can be inconsistent between different officers. Machine Learning can learn patterns from past decisions and provide a fast, data-driven first screening of every new application.

Loan Approval Predictor is a beginner-friendly ML project that predicts whether a loan application is likely to be **approved or rejected**. It trains and compares three classification models (Logistic Regression, Decision Tree and Random Forest), selects the best one and exposes it through a simple **Streamlit** web interface where a user enters applicant details and instantly receives a prediction with an approval probability.

2. Problem Statement

To build a binary classification system that, given applicant details (gender, marital status, dependents, education, employment type, applicant and co-applicant income, loan amount, loan term, credit history and property area), predicts the target *Loan_Status* (Approved = Y, Rejected = N) and presents the result through an easy-to-use web application.

Objectives

- Prepare and clean a loan application dataset for machine learning.
- Train and compare multiple classification algorithms using standard evaluation metrics.
- Save the best model and integrate it into an interactive Streamlit application.
- Document the design, implementation, testing and learnings of the project.

3. Functional Requirements

- **FR1 - Data input:** The user can enter 11 applicant attributes through form fields (drop-downs and number inputs).
- **FR2 - Prediction:** On clicking *Predict Loan Status*, the system returns Approved or Rejected.
- **FR3 - Probability:** The system displays the approval probability as a percentage and a progress bar.
- **FR4 - Model information:** The app shows the model name and its accuracy, precision, recall and F1-score.
- **FR5 - Training pipeline:** A script loads the dataset, cleans it, trains three models, compares them and saves the best one.
- **FR6 - Input validation:** Numeric inputs are restricted to valid ranges; categorical inputs are restricted to fixed choices.

4. Non-functional Requirements

- **Usability:** Simple two-column form usable by a non-technical person.
- **Performance:** A prediction is returned in well under a second; the model is cached after the first load.
- **Portability:** Runs on any system with Python 3.9+ and the packages listed in requirements.txt.
- **Maintainability:** Training, data generation and UI are in separate, commented files.
- **Reproducibility:** Fixed random seeds (42) give the same dataset, split and results every time.
- **Reliability:** The OneHotEncoder is set to ignore unseen categories so the app does not crash on new values.

5. System Architecture

The system has two phases. In the **training phase**, *train_model.py* reads the dataset, cleans and encodes it, trains three models, evaluates them on a held-out test set and saves the best scikit-learn pipeline as *model.pkl*. In the **prediction phase**, the Streamlit app loads this pipeline, converts user input into a one-row DataFrame and returns the predicted class and probability. Because the preprocessing is part of the saved pipeline, the same transformations are applied during training and prediction.

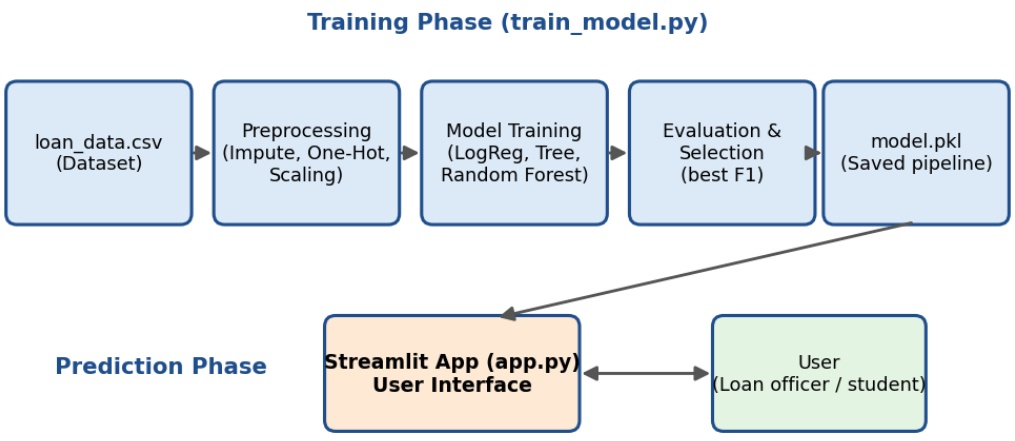


Figure 1: System architecture

6. Design Diagrams

6.1 Use Case Diagram

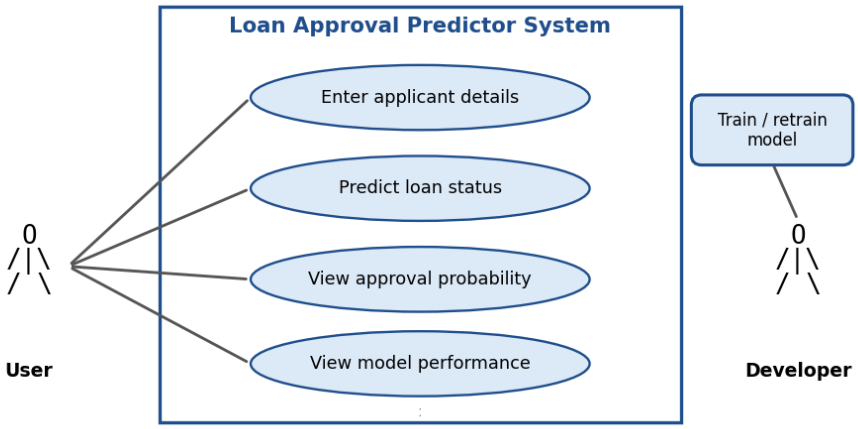


Figure 2: Use case diagram (User and Developer actors)

6.2 Workflow Diagram

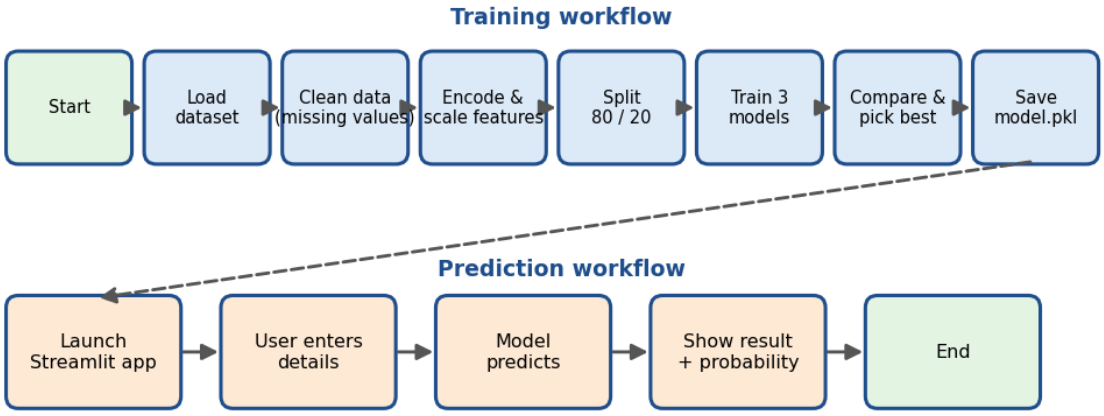


Figure 3: Training and prediction workflow

6.3 Sequence Diagram

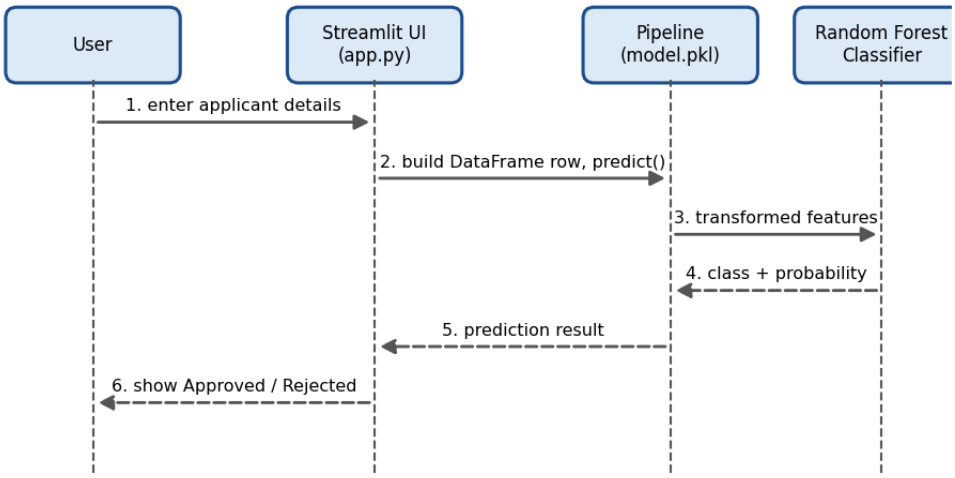


Figure 4: Sequence of a single prediction request

6.4 Class / Component Diagram

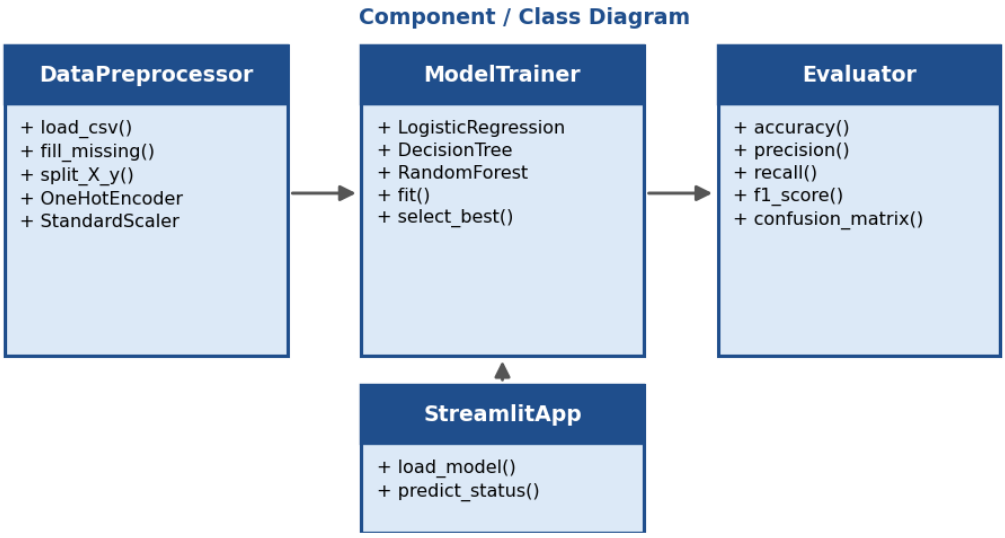


Figure 5: Components of the system

An ER diagram is not included because the project does not use a database; the dataset is stored in a CSV file.

7. Design Decisions and Rationale

Decision	Rationale
Binary classification	The outcome is either approved or rejected, so a classifier fits the problem directly.
Three models compared	Logistic Regression is a simple baseline, Decision Tree is easy to interpret, Random Forest is robust and handles non-linear patterns. Comparing them avoids guessing.
Select best by F1-score	The classes are imbalanced (about 69% approved), so accuracy alone can mislead; F1 balances precision and recall.
Scikit-learn Pipeline + ColumnTransformer	Keeps preprocessing and model together so the app applies identical transformations and cannot have training/serving mismatch.
Streamlit for the UI	Pure-Python, fast to build and ideal for ML demos without HTML/JS knowledge.
joblib for saving the model	Standard, efficient way to persist scikit-learn objects.
Synthetic dataset with a real-dataset layout	Allows a fully reproducible, self-contained project. The column layout matches the public Kaggle Loan Prediction dataset so real data can be dropped in.

8. Implementation Details

Technology stack: Python, Pandas, NumPy, Scikit-learn, Matplotlib, Joblib and Streamlit.

Files:

- **generate_dataset.py** - creates data/loan_data.csv (1000 records, 12 features + target) using fixed random seed 42; approval depends mainly on credit history and the EMI-to-income ratio, with random noise.
- **src/utils.py** - shared helpers: loads and cleans the data (mode for categorical and median for numeric missing values), builds the ColumnTransformer (OneHotEncoder for categorical, StandardScaler for numeric columns), loads the saved model and provides predict_loan().
- **train_model.py** - splits the data 80/20 with stratification, trains the three models, calculates metrics, saves the best pipeline to models/model.pkl, the metrics to models/metrics.json and the plots to docs/screenshots/.
- **app.py** - Streamlit interface; loads the model with caching, collects the inputs, calls predict_loan() and displays the result with the approval probability.
- **tests/test_model.py** - five automated tests; **requirements.txt**, **README.md**, **statement.md** and **start_loan_predictor.bat** - documentation and a one-click Windows launcher.

Dataset summary: 1000 applications, 690 approved (Y) and 310 rejected (N). Training set: 800 records, test set: 200 records.

Note on the dataset: The dataset is synthetic and was generated by generate_dataset.py. Results therefore reflect the patterns built into this data and should not be treated as real banking performance. The same code runs on the real Kaggle dataset by replacing loan_data.csv.

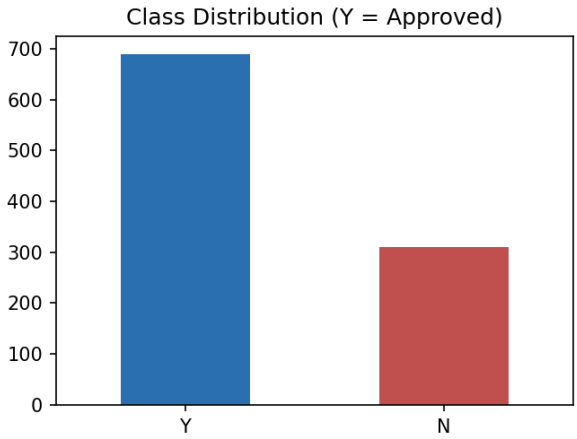


Figure 6: Class distribution of the target variable

9. Screenshots / Results

All models were evaluated on the same 20% test set (200 records). **Random Forest** was selected automatically because it achieved the highest F1-score.

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	86.5%	89.4%	91.3%	90.3%
Decision Tree	84.0%	85.3%	92.8%	88.9%
Random Forest (selected)	86.5%	85.8%	96.4%	90.8%

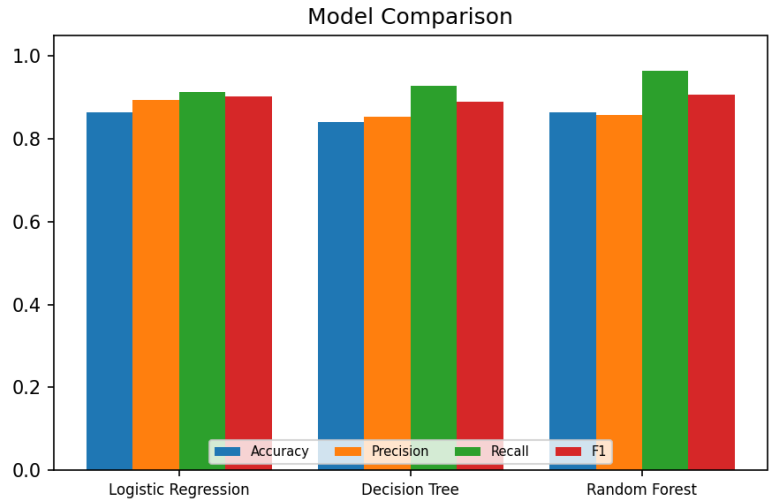


Figure 7: Model comparison

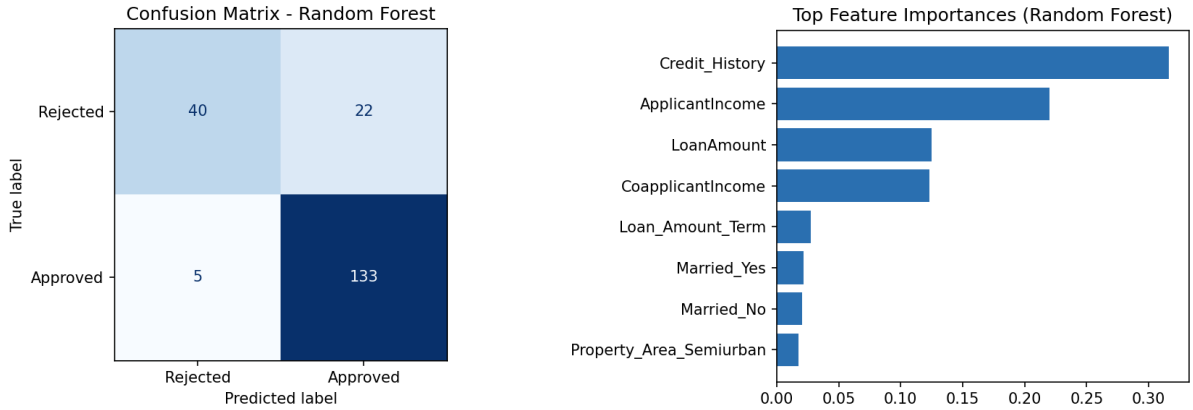


Figure 8: Confusion matrix (left) and top feature importances (right)

Observation: Credit history is the most important feature (about 32% importance), followed by applicant income, loan amount and co-applicant income. This agrees with real-world lending practice.

10. Testing Approach

Testing was done at three levels:

- **Model testing:** 80/20 stratified train-test split; accuracy, precision, recall, F1-score and confusion matrix on the unseen test set.
- **Functional testing:** sample applicants with known characteristics were passed through the saved pipeline, exactly as the app does (results below); in addition, five automated tests in tests/test_model.py pass.
- **Input/robustness testing:** number inputs have minimum values, categorical inputs are drop-downs, and the encoder ignores unseen categories.

#	Test input	Predicted	P(approve)
1	Good credit, 5000 + 1500 income, loan 120k, 360 months, Graduate, Urban	Approved	85.4%
2	Poor credit, income 2000, loan 500k, Not Graduate, Rural	Rejected	19.1%
3	Good credit, high income 12000, loan 150k, Semiurban	Approved	96.5%
4	Good credit but very low income 1500 and loan 400k	Rejected	39.9%
5	Poor credit but high income 15000, loan 100k	Approved	52.2%

The outputs match expectations: applicants with good credit and affordable loans are approved with high probability, while an unaffordable loan or poor credit history lowers the probability sharply. Case 5 is a borderline decision (about 52%), showing that a very high income can partly, but not fully, offset a poor credit history.

11. Challenges Faced

- **Class imbalance:** The first version of the data had too many approvals, making accuracy look artificially high. The data was rebalanced to about 69% approvals and models were compared with F1-score as well.
- **Categorical data:** Machine learning models need numbers, so one-hot encoding was added inside a pipeline to avoid mismatch between training and prediction.
- **Dataset availability:** A clean dataset was needed for a reproducible project, so a synthetic dataset with the standard Loan Prediction column layout was generated.
- **Project submission:** The first GitHub repository contained only the README and licence; the complete code, dataset, model and report were added afterwards.

12. Learnings and Key Takeaways

- How a complete ML workflow works: data cleaning, encoding, training, evaluation, saving and deployment.
- Why accuracy alone is not enough and how precision, recall, F1-score and the confusion matrix give a fuller picture.
- Using scikit-learn Pipelines and ColumnTransformer to keep preprocessing consistent.
- Comparing models instead of assuming one is best; Random Forest gave the best F1 here but Logistic Regression was almost as good and is easier to explain.
- Building a simple, interactive front-end with Streamlit and caching the model for speed.
- Importance of documentation (README, report) and a clean repository structure.

13. Future Enhancements

- Train on a real-world bank dataset and handle class imbalance with SMOTE or class weights.
- Hyperparameter tuning using GridSearchCV or RandomizedSearchCV and cross-validation.
- Explainable AI with SHAP or LIME to show why a loan was approved or rejected.
- Add fairness checks so the model does not discriminate on gender or marital status.
- Deploy the app online using Streamlit Community Cloud and store predictions in a database.
- Try gradient boosting models such as XGBoost or LightGBM.

14. References

- Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, 12, 2011.
- Scikit-learn documentation - https://scikit-learn.org/stable/
- Streamlit documentation - https://docs.streamlit.io/
- Pandas documentation - https://pandas.pydata.org/docs/
- Kaggle - Loan Prediction Problem Dataset (column layout reference) - https://www.kaggle.com/
- Breiman, L., *Random Forests*, Machine Learning, 45(1), 5-32, 2001.
- Project repository - https://github.com/mohit939926-a11y/AI-ML-vityarthi-project